

QuizLoop.ai: A Gemma 4 Learning Loop for Evidence-Based Studying

QuizLoop.ai is an iOS-first, quiz-first tutor for the Future of Education track. The project started from a classroom observation: students like using AI, but chat is often the wrong interface for learning. A student has to know what to ask, how much context to paste, how to test themselves, and how to decide whether they actually understand the material. QuizLoop changes that interaction. The student gives the app learning material, and the system turns it into a guided sequence of checks, feedback, and follow-up quizzes.

Product Idea

The core loop is simple: save source material, let **Gemma 4** decompose it, quiz the learner, grade the answers, store the evidence, and build the next quiz from that history. The interface is intentionally small: Home, Library, and History. Library holds notes from pasted text, Wikipedia, books, and math templates. Home keeps one active learning journey in focus. History lets the learner review prior quiz attempts and feedback.

This is not a chatbot with quiz features attached. It is a structured learning system that uses Gemma as an intelligence layer inside a bounded product loop.

Technical Implementation

The product architecture is centered on the SwiftUI iOS app. The iOS app owns the learning interface, local **SQLite** memory, quiz history, and a ``GemmaService`` boundary that lets the same learning loop run against different Gemma runtimes. A companion progressive web app mirrors the same framework for the public video demo. The deployed web version uses Cloudflare Pages and API functions, while local development can run the same API shape with Node. The memory layer is **SQLite**-oriented: the iOS/local app uses **SQLite** directly, and the hosted demo uses Cloudflare D1 for the same relational pattern.

The main objects are:

- ``Note``: raw source text, title, source type, processing state, and Gemma summary.
- ``Topic``: a high-level area extracted from the note.
- ``Segment``: the smallest learnable concept, anchored to source evidence.
- ``Question``: prompt, answer, choices, type, topic, subtopic, segment reference, importance, difficulty, and canonical concept key.
- ``Attempt``: the learner's response, score, feedback, timestamp, and missed ideas.
- ``QuizSession``: the completed quiz, grouped attempts, score, and note relationship.
- ``UserAction``: interface events that help the system understand how the learner moved through the journey.

These objects matter because they keep the system from becoming a random worksheet generator. Questions are durable learning objects. Attempts are evidence. Quiz sessions are history. **SQLite** gives Gemma structured memory instead of asking the model to infer everything from a chat transcript.

How Gemma 4 Is Used

Gemma 4 is used for bounded jobs, not open-ended conversation.

During note processing, Gemma receives only the note text and returns structured JSON: topics, concepts, grounded questions, correct answers, distractors, importance, difficulty, and source references. The prompt asks Gemma to stay inside the note, avoid meta questions, and create checks that actually test understanding.

During quiz expansion, Gemma receives **SQLite** context: previous questions, recent attempts, weak concepts, mastered concepts, selected focus, and relevant source excerpts. This lets the next quiz avoid repeats, target weak areas, and introduce adjacent or harder concepts after strong performance.

During grading, multiple-choice answers are graded deterministically. Open-ended answers are routed through Gemma when available, using the saved answer, source evidence, and rubric. The goal is semantic grading: whether the learner expressed the idea, not whether they matched the exact wording.

Every Gemma output is parsed, validated, and stored before it appears in the UI. The app rejects duplicate prompts, overlapping answer choices, unsupported facts, answer-revealing wording, and weak distractors. This validation layer became one of the most important parts of the project.

Quiz Planner

QuizLoop builds a starter quiz bank as soon as a note is saved, then expands the bank in the background. The quiz planner selects from stored questions using understanding score, last-seen time, prior misses, difficulty, importance, and optional focus. A perfect quiz should move the learner forward into adjacent or harder concepts. A missed concept should return later in a different form.

Quiz size is adaptive instead of fixed. Short notes can produce smaller quizzes, while larger notes and books can produce longer sessions. The planner also blocks near-duplicate questions inside the same quiz by comparing canonical concept keys, answer overlap, subtopic repetition, and prompt families.

iOS and Google AI Edge Path

The SwiftUI app is the product-ready direction. It uses a `GemmaService`` interface so the learning system does not depend on one model runtime. During development, the app can use an **Ollama**-compatible Gemma endpoint. For offline production use, the same interface is structured to load a bundled Gemma model through the Google AI Edge / MediaPipe runtime path.

This matters because QuizLoop is meant to be a private learning assistant. The native app keeps notes, questions, attempts, scores, and feedback in local **SQLite**. Gemma receives only the structured context needed for the current learning task. With the on-device runtime path, the phone can become the student's self-contained study system instead of a thin client for a cloud chatbot.

Validation

Testing focused on repeated quiz loops rather than one-off prompts. I used topics such as Photosynthesis, Java basics, LeBron James, Baltimore Ravens, Cleveland Cavaliers, Steve Jobs, Google, clock-hand angle problems, and Pythagorean theorem practice.

The most important bugs were educational, not just technical: repeated questions, shallow distractors, unsupported facts, quiz banks getting stuck, and progress that did not reflect the learner's answers. These failures pushed the system toward canonical concept keys, background quiz expansion, source-grounded validation, history-aware selection, and better quiz reports.

Impact

QuizLoop.ai reimagines AI tutoring as an evidence loop. The note is the curriculum. Gemma creates and grades bounded checks. **SQLite** remembers the learner's evidence. The interface keeps the student moving

forward without requiring prompt engineering or manual study planning.

For students, this lowers friction. For educators, it suggests a safer classroom pattern: AI can be powerful when it is constrained by source material, structured memory, and measurable learning actions.

Project Links

- Live demo: <https://quizloop.ai>
- Code repository: <https://github.com/surpol/QuizLoop>
- Gemma endpoint used for the web demo: <https://gemma-quizloop.suryapolina.com>
- Local desktop model: `gemma4:e2b` through **Ollama**
- iOS edge path: SwiftUI + **SQLite** + `GemmaService` + Google AI Edge / MediaPipe runtime path for a bundled Gemma model